NetroTrack Production — Errors & Troubleshooting Runbook

Every major deployment/runtime/load-balancer issue encountered and how it was resolved

1. Incident: Docker socket permission denied

```
docker compose ps
permission denied while trying to connect to the Docker API at unix:///var/run/docker.sock
```

Resolution used: continue in the VM session and use the account/configuration that has Docker access. After access was corrected, docker compose ps worked normally. No need to exit the VM.

2. Incident: backend restart loop — Prisma/OpenSSL

```
PrismaClientInitializationError:
Unable to require(...libquery_engine-debian-openssl-1.1.x.so.node)

Prisma cannot find the required libssl system library.
Details: libssl.so.1.1: cannot open shared object file

Prisma failed to detect the libssl/openssl version and defaulted to openssl-1.1.x.
```

Root cause recorded in the logs: the Docker image used node:20-bookworm-slim and OpenSSL was not available during Prisma client generation/runtime. Prisma therefore selected an incompatible OpenSSL 1.1 query engine.

Fix: install OpenSSL in both the builder and runner stages of the Dockerfile, rebuild the image, publish the production image, then redeploy.

```
# Recommended Dockerfile pattern for Debian Bookworm slim

FROM node:20-bookworm-slim AS builder

RUN apt-get update     && apt-get install -y --no-install-recommends openssl     && rm -rf /var/lib/apt/lists/*

# ... install dependencies / generate Prisma / build ...


FROM node:20-bookworm-slim AS runner

RUN apt-get update     && apt-get install -y --no-install-recommends openssl     && rm -rf /var/lib/apt/lists/*

# ... copy built application and start ...
```

After the corrected image was deployed, the backend remained UP instead of restarting.

3. Incident: localhost:3000 from the VM failed while container was healthy

```
curl -i http://127.0.0.1:3000
curl: (7) Failed to connect to 127.0.0.1 port 3000
```

At that time docker compose ps showed the backend as Up but only exposed port 3000 internally. The Compose configuration was then changed so the backend exposed/published port 3000 for the intended load-balancer path.

```
backend:
  ...
  expose:
    - "3000"
```

Important distinction: expose makes the port available to linked Docker services/networks; host-level curl to 127.0.0.1:3000 requires a host port mapping such as 127.0.0.1:3000:3000 or 0.0.0.0:3000:3000. The final working setup used a host-reachable port 3000 for the LB backend path.

4. Incident: curl/wget missing inside backend container

```
docker compose exec backend sh
curl -i http://127.0.0.1:3000
sh: 1: curl: not found

wget -qO- http://127.0.0.1:3000
sh: 2: wget: not found
```

Resolution: do not add troubleshooting packages to the production image just for a one-off test. Node.js itself can test HTTP:

```
docker compose exec backend node -e "require('http').get('http://127.0.0.1:3000/health', r => { console.log('HTTP', r.statusCode); r
```

5. Incident: root URL returned 404

```
HTTP 404
Cannot GET /
```

This was not a failure. The application exposes /health. The compiled application was inspected and showed the health route under /app/apps/backend/dist/app.js.

```
app.use('/health', (_req, res) => {
  res.status(200).json({ success: true, message: 'Server is healthy' });
});
```

6. Incident: gcloud from VM failed with insufficient authentication scopes

```
ERROR: (gcloud.compute.instances.describe)
Could not fetch resource:
- Request had insufficient authentication scopes.
```

Resolution: run infrastructure-level gcloud commands from the local workstation where the appropriate Google credentials/scopes are available, rather than depending on the VM's attached service-account scopes.

7. Incident: attempted GET command in zsh

```
GET http://10.128.0.4:3000/health
zsh: command not found: GET
```

Resolution: use curl, not HTTP verb text as a shell command:

```
curl -i http://10.128.0.4:3000/health
```

8. Incident: duplicate backend group

```
gcloud compute backend-services add-backend ...
ERROR:
Duplicate instance groups in backend service.
```

Resolution: inspect the backend service before adding anything. The instance group was already attached, so the command was unnecessary. get-health then showed the group HEALTHY.

```
gcloud compute backend-services describe netrotrack-backend-service   --project=netro-track-prod --global   --format="yaml(name,prot
```

9. Incident: initial HTTPS curl returned SSL_ERROR_SYSCALL

```
curl -i https://netro-track-api.netrofusion.in/health
curl: (35) LibreSSL SSL_ERROR_SYSCALL
```

Diagnosis performed in layers: DNS → TCP 443 → certificate → TLS handshake → HTTP/2 request. DNS resolved to 8.232.197.48, TCP 443 connected, openssl showed a valid Google Trust Services certificate, and the final curl request returned HTTP/2 200. The issue was therefore transient rather than a persistent certificate/proxy configuration error.

```
nc -vz 8.232.197.48 443
```

```
openssl s_client   -connect netro-track-api.netrofusion.in:443   -servername netro-track-api.netrofusion.in

curl -v --tlsv1.2 https://netro-track-api.netrofusion.in/health
```

10. Incident: HTTP redirect initially returned 404

```
curl -I http://netro-track-api.netrofusion.in/health
HTTP/1.1 404 Not Found
```

The original HTTP forwarding path was replaced with a dedicated redirect URL map. The correct URL-map import YAML was:

```
name: netrotrack-http-redirect-map

defaultUrlRedirect:
  httpsRedirect: true
  stripQuery: false
```

After the redirect proxy and forwarding rule were recreated, the final result became 301 with Location: https://netro-track-api.netrofusion.in:443/health, followed by HTTP/2 200.

11. Incident: invalid gcloud redirect flags

```
gcloud compute url-maps create netrotrack-http-redirect-map   --project=netro-track-prod   --default-redirect   --https-redirect

ERROR: unrecognized arguments:
--default-redirect
--https-redirect
```

Resolution: use URL-map YAML import instead of unsupported create flags.

12. Incident: URL-map import rejected 'kind'

```
ERROR:
Additional properties are not allowed ('kind' was unexpected)
```

Resolution: remove the exported/API-only 'kind' property and import only the supported fields. The minimal YAML above succeeded.

13. Incident: target HTTP proxy created before URL map existed

```
ERROR:
The resource 'projects/netro-track-prod/global/urlMaps/netrotrack-http-redirect-map'
was not found
```

Resolution: create/import the URL map first, verify it, then create the target HTTP proxy.

14. Correct diagnostic order for future incidents

```
1. docker compose ps
2. docker compose logs --tail=100 backend
3. curl -i http://127.0.0.1:3000/health       # host mapping exists
4. node HTTP test inside container if curl/wget absent
5. inspect /health route in dist
6. verify VM tag + firewall
7. verify instance group + named port
8. verify backend service
9. get backend health
10. verify DNS -> reserved IP
11. verify forwarding rules
12. verify target proxy -> URL map
13. verify certificate ACTIVE
14. openssl s_client
15. curl -IL HTTP and curl -I HTTPS
```